

Terminal Emulator Research Report

For Velocity: A Blazing-Fast File Manager for macOS

Date: March 7, 2026

Purpose: Inform the embedded terminal product roadmap

Scope: Leading terminal emulators, developer preferences,
performance analysis, and embedded terminal best practices

PART 1: LEADING TERMINAL EMULATORS - COMPETITIVE LANDSCAPE

1.1 iTerm2 (macOS)

Status: The long-reigning macOS terminal standard. Free, open source, macOS-only.

Key Differentiating Features:

- Deep tmux control mode integration (tmux -CC renders panes as native tabs/splits -- widely considered its killer feature)
- Python scripting API for full automation (replaced AppleScript API)
- Triggers: user-defined regex-matched actions on terminal output
- Shell integration with semantic prompt awareness (marks, navigation between commands)
- Instant Replay: travel backward in time through terminal output
- Built-in password manager (macOS Keychain-backed)
- Broadcast input to multiple panes simultaneously
- Inline images (proprietary iTerm2 image protocol)
- Captured Output: IDE-like error/warning detection with editor integration
- Automatic Profile Switching based on hostname/username/directory
- Badges: overlay user/host/branch info on terminal background
- Global search across all tabs simultaneously
- Advanced paste with base64 conversion and character transformation
- Annotations on selected terminal text
- Hotkey window (dropdown terminal)

Performance Characteristics:

- No GPU acceleration -- uses CPU rendering
- Memory: ~180MB for a single tab (vs 40MB for Terminal.app)
- With 20+ tabs: 2-3GB RAM
- Renderer struggles with high-throughput output (tailing busy logs)
- Measurably higher input latency than GPU-accelerated alternatives

What Developers Love:

- Most feature-complete terminal on macOS, period
- tmux integration is unmatched -- converts tmux sessions to native UI
- Profile system (per-host color schemes, fonts, env vars, startup commands)
- Regex search through entire scrollbar
- Rock-solid stability after 20+ years of development
- Free forever

What Developers Hate:

- Resource hog (RAM and CPU)
- No GPU rendering = sluggish under heavy output
- No file-based configuration (requires GUI prefs panel)
- macOS-only (no cross-platform)
- Increasingly seen as "bloated" by minimalist users

Relevance to Velocity: iTerm2's tmux integration, shell integration marks, triggers, and profile system are gold-standard features. Its resource overhead is the cautionary tale - our embedded terminal must be lightweight.

1.2 Ghostty (macOS, Linux)

Status: Newest major entrant (public release December 2024). Created by Mitchell Hashimoto (HashiCorp co-founder). Written in Zig. Reached 45,247 GitHub stars by March 2026.

Key Differentiating Features:

- GPU-accelerated via Metal (macOS) and OpenGL (Linux) -- only Metal-based terminal supporting ligatures without CPU fallback
- Platform-native UI: tabs and splits rendered with native macOS components
- Zero-configuration philosophy: works perfectly out of the box with Nerd Fonts
- Kitty Graphics Protocol support for inline images
- Kitty Keyboard Protocol for improved key handling
- Quick Terminal (dropdown that animates from menu bar)
- macOS-specific: proxy icon in title bar, Quick Look, force touch, Secure Keyboard Entry with animated lock indicator
- Synchronized rendering for flicker-free output
- Automatic light/dark mode theme switching based on system preference
- Hundreds of built-in themes selectable via single config line
- Proper grapheme clustering for multi-codepoint emoji and RTL scripts
- Shell integration system supporting Bash, Zsh, Fish, Elvish, and Nushell
- State recovery on restart

Performance Characteristics:

- Among the fastest GPU-accelerated terminals
- Metal rendering on macOS gives excellent font rendering via Core Text
- Low memory footprint (comparable to Alacritty's ~30MB tier)
- Real-world latency effectively matches Alacritty despite higher benchmark numbers
- Smooth scrolling with minimal CPU usage

What Developers Love:

- "Feels like a Mac app" -- native integration is exceptional
- Works perfectly out of the box, zero config needed

- Faster and lighter than iTerm2 with most of the features people actually use
- Font rendering quality on macOS is among the best
- Active development by a respected developer

What Developers Hate:

- No Windows support
- Configuration is file-only (no GUI preferences panel)
- Fewer features than iTerm2 (no tmux control mode, no Python API, no triggers)
- Relatively young -- some features still in progress
- Some developers have switched back to Kitty after finding Ghostty's feature set insufficient

Relevance to Velocity: Ghostty proves that native macOS integration + GPU rendering + sensible defaults is the winning formula. Our terminal should match its "zero config, just works" philosophy and its native-feeling UI.

1.3 Kitty (macOS, Linux, BSD)

Status: Mature GPU-accelerated terminal. Written in C and Python. 30% market share among Arch Linux users (January 2025).

Key Differentiating Features:

- GPU rendering + SIMD vector CPU instructions for best-in-class throughput
- Kitty Graphics Protocol (the most widely adopted terminal image standard)
- Kitty Keyboard Protocol (more accurate key event reporting)
- "Kittens" plugin system: extensible via Python scripts
- Remote control protocol: control Kitty from external scripts/programs
- Built-in terminal multiplexer (tabs, splits, layouts)
- Extensive built-in kittens: SSH, diff, Unicode input, hints (URL/path picking), broadcast
- Threaded rendering: child process I/O in separate thread from rendering
- Glyph caching in VRAM for zero font rendering bottleneck
- SIMD-accelerated byte stream parsing

Performance Characteristics:

- Throughput: 134.55 MB/s average -- approximately DOUBLE the next fastest terminal
- CPU usage during scrolling: 6-8% (vs 15-17% GNOME Terminal, 29-31% Konsole)
- Configurable latency tuning: `input_delay`, `repaint_delay`, `sync_to_monitor`
- Minimal latency config: `input_delay=0`, `repaint_delay=2`, `sync_to_monitor=no`
- Default scrollbar: 2000 lines (configurable)

What Developers Love:

- Fastest throughput of any terminal, period
- Image protocol is becoming the de facto standard
- Kittens system is incredibly powerful (SSH kitten, diff kitten, etc.)
- Remote control protocol enables advanced automation

- Not bloated despite extensive features
- Cross-platform (macOS + Linux)

What Developers Hate:

- Does not support Sixel (only its own graphics protocol)
- Creator's sometimes abrasive communication style
- OpenGL only (no Metal on macOS)
- Configuration syntax is non-standard
- Some users find it less "Mac-native" feeling than Ghostty or iTerm2

Relevance to Velocity: Kitty's performance architecture (VRAM glyph caching, SIMD parsing, threaded rendering) is the gold standard for throughput. Supporting the Kitty Graphics Protocol would enable image previews inside our terminal. Its kitten/plugin concept could inspire our module system.

1.4 Alacritty (Cross-platform)

Status: Minimalist GPU-accelerated terminal. Written in Rust. Pioneered the GPU terminal movement.

Key Differentiating Features:

- Fastest input latency of any terminal in benchmarks
- Deliberately minimal: no tabs, no splits, no built-in multiplexer
- Philosophy: be the best rendering engine, delegate everything else to tmux/Zellij
- Vi mode for keyboard-driven scrollback navigation and selection
- TOML configuration file
- ~30MB RAM footprint -- smallest in the GPU-accelerated tier
- Cross-platform: macOS, Linux, BSD, Windows

Performance Characteristics:

- Lowest input latency in hardware benchmarks
- GPU rendering via OpenGL
- Second fastest throughput (54 MB/s, behind Kitty's 134 MB/s)
- Extremely low resource consumption
- Fast startup

What Developers Love:

- Pure speed -- typing feels instant
- No bloat, no unnecessary features
- Pairs perfectly with tmux or Zellij
- Cross-platform consistency
- Tiny memory footprint

What Developers Hate:

- No tabs or splits (must use external multiplexer)

- No image protocol support (no Sixel, no Kitty Graphics)
- No font ligature support (deliberate design choice)
- No shell integration
- Minimal feature set frustrates users who don't want tmux
- YAML config was replaced with TOML, causing migration friction

Relevance to Velocity: Alacritty proves that raw rendering performance matters deeply. Our embedded terminal should aspire to Alacritty-level input latency while offering the integrated features (tabs, splits) that Alacritty deliberately omits.

1.5 Warp (macOS, Linux, Windows)

Status: AI-powered "Agentic Development Environment." Written in Rust. Venture-funded. TIME Best Inventions 2025.

Key Differentiating Features:

- Block-based output: each command+output is a discrete, selectable, sharable unit
- Warp AI: natural language command generation, error explanation, task automation
- Four AI interaction modes: traditional CLI, AI completions, interactive chat, autonomous/agentic
- IDE-like text editor for command input (mouse cursor, selection, multi-line editing)
- Smart command completions (hundreds of commands)
- Warp Drive: cloud-based sharable workflows, notebooks, runbooks
- Team collaboration: shared workflows, session sharing, block sharing with permanent links
- Command palette for quick navigation
- Rich history with exit codes and timestamps
- Secret redaction for API keys
- Launch configurations for saved sessions
- Warp Code: built-in file editor with agent code review

Performance Characteristics:

- Rust-based rendering with GPU acceleration
- Competitive performance in throughput benchmarks (mixed results vs minimalist terminals)
- Additional overhead from AI features and block-based UI
- Cross-platform support including Windows

What Developers Love:

- AI command suggestions are genuinely useful (like having a senior dev available)
- Block-based output transforms how you interact with command history
- Modern, polished UX -- feels like software from 2026, not 1970
- Team sharing features for workflows and runbooks
- Excellent onboarding for terminal newcomers

What Developers Hate:

- Requires account creation and telemetry (privacy concerns)

- AI features require internet connection
- Not open source (freemium model)
- Departs significantly from traditional terminal UX -- muscle memory breaks
- Resource usage higher than minimalist alternatives
- Some developers distrust the business model

Relevance to Velocity: Warp's block-based output model is genuinely innovative and worth studying. The command palette concept aligns with our own Cmd+Shift+P vision. However, the mandatory account/telemetry model is an anti-pattern we should avoid.

1.6 WezTerm (Cross-platform)

Status: Feature-rich GPU-accelerated terminal and multiplexer. Written in Rust. Open source (MIT).

Key Differentiating Features:

- Lua scripting for configuration (hot-reloadable, Turing-complete)
- Built-in multiplexer replacing tmux for many users
- Broadest graphics protocol support: Sixel, Kitty Graphics, AND iTerm2 inline images
- SSH client with multiplexing support
- Serial port support (Arduino, embedded devices)
- Bundles JetBrains Mono, Nerd Font Symbols, and Noto Color Emoji by default
- Full font shaping with ligature support
- Workspace and session management
- Dynamic tab titles, custom key tables via Lua
- Cross-platform: macOS, Linux, Windows, BSD

Performance Characteristics:

- GPU-accelerated rendering
- Higher latency than Alacritty/Kitty in benchmarks
- Variable performance across different test scenarios
- Good memory efficiency

What Developers Love:

- Lua scripting is a game-changer for power users (automate anything)
- Only terminal supporting ALL major image protocols
- Built-in multiplexer means no tmux dependency
- Cross-platform consistency (same config everywhere)
- Excellent font handling out of the box
- SSH and serial port support built-in

What Developers Hate:

- Higher latency than top-tier performers
- Lua config can be complex for simple changes

- Less "native" feeling on macOS than Ghostty or iTerm2
- Some rendering inconsistencies across platforms
- Development pace has slowed compared to Ghostty

Relevance to Velocity: WezTerm's multi-protocol image support (Sixel + Kitty + iTerm2) is the most inclusive approach. Its Lua scripting model demonstrates how configuration-as-code unlocks power-user workflows. Built-in SSH and serial support are features our terminal could benefit from.

1.7 Hyper (Cross-platform)

Status: Electron-based terminal by Vercel. Written in HTML/CSS/JavaScript.

Key Differentiating Features:

- Built on web technologies (HTML, CSS, JavaScript)
- Rich plugin ecosystem via npm
- Beautiful default UI
- Customizable via ~/.hyper.js
- Tabs and split panes

Performance Characteristics:

- Heavy resource usage (Electron overhead)
- Noticeably higher latency than native terminals
- Performance degrades with plugins
- With 5+ windows alongside other Electron apps, system becomes unresponsive

What Developers Love:

- Beautiful out of the box
- Easy customization via web technologies
- Familiar tech stack for web developers
- Good plugin ecosystem

What Developers Hate:

- Slow, sluggish, resource-hungry
- Electron = high memory baseline
- Performance degrades quickly with plugins
- Not competitive with native terminals on any performance metric

Relevance to Velocity: Hyper is the anti-pattern. It proves that web technologies are wrong for terminal emulation. Our native Swift + Metal approach is the correct architecture. Hyper's beautiful defaults and easy plugin system are worth studying for UX, but not for implementation.

1.8 Terminal.app (macOS built-in)

Status: Apple's built-in terminal. Ships with every Mac.

Key Differentiating Features:

- Zero installation required
- Deep macOS integration (Services menu, Automator)
- VoiceOver accessibility support
- Extremely low resource usage (~40MB per tab)
- Secure Keyboard Entry
- Touch Bar support (on applicable hardware)

Performance Characteristics:

- Very low memory: ~40MB per tab
- No GPU acceleration
- Excels in dense cell rendering but struggles with scrolling
- Good enough for basic use

What Developers Love:

- Always available, no installation needed
- Very low resource usage
- Reliable and stable
- Good enough for basic tasks

What Developers Hate:

- No split panes
- Limited customization
- No advanced search
- No image protocol support
- No shell integration features
- Falls behind on modern terminal standards

Relevance to Velocity: Terminal.app's low resource consumption (40MB) is the floor we should target. Users who outgrow Terminal.app are our target market.

1.9 Windows Terminal (Windows)

Status: Microsoft's modern terminal. Open source.

Key Differentiating Features:

- Tabs with profiles for CMD, PowerShell, WSL distributions
- GPU rendering via DirectWrite/DirectX
- JSON-based configuration
- Customizable themes, backgrounds, transparency
- Unicode and UTF-8 support including emoji

- Split panes

Performance Characteristics:

- DirectX GPU acceleration
- Smooth scrolling and crisp text
- Lower CPU usage than legacy terminals
- Good performance consistency under load

Relevance to Velocity: Windows Terminal shows that even Microsoft recognized GPU rendering and tabs are non-negotiable. Its JSON config and profile system are well-designed.

1.10 Tabby (formerly Terminus)

Status: Cross-platform SSH-focused terminal. Electron-based. Open source.

Key Differentiating Features:

- Integrated SSH client and connection manager (its primary value proposition)
- Encrypted container for SSH secrets
- X11 and port forwarding
- Automatic jump host management
- Serial terminal support
- Zmodem file transfers
- Plugin ecosystem (Docker, Telnet, quick commands)
- Fully configurable shortcuts with multi-chord support

Performance Characteristics:

- Electron-based: noticeably less responsive during heavy scrolling
- Higher resource usage than native terminals
- Acceptable for SSH-focused workflows

Relevance to Velocity: Tabby's SSH connection manager and encrypted secret storage are worth studying for our remote connectivity features. Its Electron performance penalty confirms our native approach.

1.11 Rio (macOS, Linux)

Status: Newer Rust-based terminal using WebGPU. Active development.

Key Differentiating Features:

- WebGPU rendering (future-proof graphics API)
- Reuses Alacritty's ANSI parser, events, and grid system
- Support for both iTerm2 and Kitty image protocols
- Sixel protocol support
- RetroArch shader support (CRT-style aesthetics)

- Toggleable Vi mode
- Font ligature support
- Wayland and X11 support on Linux

Performance Characteristics:

- WebGPU rendering (potentially faster than OpenGL in the future)
- Still in active development (v0.3.0), performance not yet mature
- ARM64 native builds available

Relevance to Velocity: Rio's use of WebGPU is forward-looking. Its support for multiple image protocols (iTerm2 + Kitty + Sixel) follows the inclusive approach.

1.12 Wave Terminal

Status: Open-source terminal with file management integration. Newer entrant.

Key Differentiating Features:

- Built-in file management: create, rename, delete files/directories
- Connected file management (wsh file operations: cat, write, append, rm, cp, ls)
- Built-in VS Code-like editor with syntax highlighting
- SSH connection manager with WSL support
- AI integration (attach directory listings to AI chat, drag-drop files to AI)
- Remote file editing and preview
- Workspace organization

Performance Characteristics:

- Not in the top tier for raw performance
- Focus is on workflow integration rather than raw speed

Relevance to Velocity: Wave Terminal is the most directly relevant competitor because it combines file management + terminal + AI. However, it appears to prioritize features over performance. We should study its file-to-terminal workflows while ensuring we maintain our performance standards.

PART 2: FEATURES DEVELOPERS VALUE MOST

2.1 Feature Priority Hierarchy

Based on analysis of developer discussions (HackerNews, Reddit r/commandline), GitHub issue tracking, survey data, and review patterns, features rank in this priority order:

Tier 1 - Dealbreakers (Must Have)

1. Low input latency -- Keystroke-to-echo under 16ms (one frame at 60fps). Under 10ms is perceived as "instant." Over 20ms is perceived as "laggy."
2. True color (24-bit) -- Universal standard. All modern terminals support this.
3. Unicode support -- Including CJK, emoji, combining characters. Broken Unicode is unacceptable.
4. Tabs -- The most requested feature that Alacritty deliberately omits (and why many developers avoid it)
5. Split panes -- Horizontal and vertical splits within the terminal
6. Search in scrollback -- With regex support. Must be fast.
7. Configurable keybindings -- Developers demand control over their shortcuts
8. Font rendering quality -- Ligature support, Nerd Fonts, clear text rendering

Tier 2 - Important (Strongly Preferred)

9. GPU rendering -- Measurably improves scrolling, output flood handling, and perceived responsiveness
10. Shell integration -- OSC 133 semantic prompt marks, working directory tracking, command navigation
11. Scrollback buffer -- Large, searchable, persistent. Defaults of 2000-10,000 lines, configurable.
12. Clickable URLs and file paths -- OSC 8 hyperlink standard. Cmd-click to open.
13. Themes and color schemes -- Easy switching, dark/light mode awareness
14. Session restore -- Recover tabs and state after restart/crash
15. Fast startup -- Under 300ms for warm launch
16. Profile management -- Per-project or per-host configurations
17. Image display -- Kitty Graphics Protocol is becoming standard; Sixel for legacy

Tier 3 - Delightful (Competitive Advantages)

18. Command palette -- Quick fuzzy search for all actions
19. AI command suggestions -- Natural language to shell command translation
20. tmux integration -- Especially control mode for native pane rendering
21. Broadcast input -- Type in multiple panes simultaneously
22. Triggers/automation -- Regex-matched actions on output
23. Inline image display -- Beyond protocols: actual image rendering in output

- 24. Password manager integration
- 25. Notifications -- Bell, activity, process completion alerts
- 26. Block-based output -- Warp's innovation of treating each command as a unit
- 27. Drag-and-drop path insertion -- Critical for file manager integration

2.2 Performance - What Developers Actually Measure

Input Latency (Keystroke-to-Echo)

This is the single most important performance metric. Developers perceive latency even when they cannot consciously identify it. Jitter (variance in latency) is worse than consistent latency.

Benchmark Results (approximate, varies by hardware):

Terminal	Typical Latency	Tier
Alacritty	~4-6ms	Best
foot	~4-6ms	Best
Ghostty	~6-8ms	Excellent
Kitty	~6-8ms	Excellent
Terminal.app	~8-12ms	Good
WezTerm	~10-14ms	Acceptable
Windows Terminal	~8-12ms	Good
iTerm2	~15-25ms	Noticeable
Hyper	~30-50ms	Poor
Tabby	~25-40ms	Poor

Key insight: Anything under 16ms (one frame) is acceptable. Under 8ms is excellent. The difference between 4ms and 8ms is not perceptible, but the difference between 10ms and 25ms is.

Throughput (MB/s of Terminal Output)

Matters for: tailing logs, running large builds, cat-ing large files, git log output.

Benchmark Results:

Terminal	Throughput (MB/s)
Kitty	134.55
GNOME Terminal	61.83
Alacritty	54.05
Others	20-50

Key insight: Kitty's threaded rendering + VRAM glyph cache + SIMD parsing achieves 2x the throughput of any competitor. This architecture should inform our design.

CPU Usage During Continuous Scrolling

Terminal	CPU %
xterm	5-7%
Kitty	6-8%
GNOME Terminal	15-17%
Konsole	29-31%

Memory Usage (Idle, Single Tab)

Terminal	Memory
Alacritty	~30MB
Ghostty	~35MB
Kitty	~50MB
Terminal.app	~40MB
iTerm2	~180MB
Hyper	~200MB+

Startup Time

Target: Under 1 second cold, under 300ms warm. GPU-accelerated terminals generally achieve this. Electron-based terminals (Hyper, Tabby) are noticeably slower.

2.3 Text and Rendering Features

Font Rendering

- Ligatures: Supported by Kitty, WezTerm, Ghostty, Tabby, Wave. Partial in iTerm2 and Hyper. Not in Alacritty (by design).
- Nerd Fonts: Ghostty supports them by default. WezTerm bundles Nerd Font Symbols. Others require manual configuration with font fallback chains.
- Font shaping: WezTerm and Kitty offer the most advanced font shaping, with configurable features per font.
- Core Text rendering (macOS): Ghostty uses Metal with Core Text for the best macOS font rendering. This is important for our SwiftTerm-based implementation.

Color Support

- True Color (24-bit): Universal across all modern terminals.
- 256-color: Universal.
- Themed color schemes: Most terminals support them. Ghostty offers hundreds built-in.

Unicode and International Text

- Grapheme clustering: Only Ghostty and Kitty handle this correctly for all edge cases (emoji sequences, variation selectors, combining characters).
- CJK double-width characters: Well-supported across modern terminals.
- RTL text: Ghostty explicitly supports RTL scripts.
- The fundamental challenge: Mapping Unicode's vast character set onto fixed-width terminal grids remains unsolved at the spec level. The text sizing protocol is an emerging solution.

Image Protocols

Three protocols exist:

Protocol	Terminals Supporting	Quality	Notes
Sixel	WezTerm, Windows Termi..	Low-Medium	1980s DEC standard. Bi..
Kitty Graphics	Kitty, WezTerm, Ghostty	High	Modern. Truecolor, tra..
iTerm2 Protocol	iTerm2, WezTerm	Medium	macOS-native. Propriet..

Key insight: WezTerm is the only terminal supporting ALL three protocols. For maximum compatibility, support Kitty Graphics Protocol (the modern standard) plus iTerm2 protocol (for macOS ecosystem).

Advanced Text Styling

- Undercurl: Supported by Kitty, WezTerm, Ghostty
- Strikethrough: Widely supported
- Colored underlines: Kitty, WezTerm, Ghostty
- Bold/Italic/Dim: Universal

2.4 Shell Integration and Semantic Features

OSC 133 Semantic Prompt Protocol

This is the single most important shell integration standard. It enables:

1. Prompt-to-prompt navigation: Jump between commands in scrollback
2. Command output selection: Select the entire output of a specific command
3. Working directory tracking: Terminal knows which directory each command ran in
4. New tab/pane CWD inheritance: Open new tab in same directory as current command
5. Exit code awareness: Terminal knows if a command succeeded or failed

How it works: Shell integration scripts inject invisible OSC 133 escape sequences at key points:

- A: Prompt start
- B: Command start (after prompt)

- C: Command executed (output start)
- D: Command finished

Supported shells: Bash, Zsh, Fish, Elvish, Nushell

Key insight: This is critical for our embedded terminal. Shell integration enables the directory sync between file panes and terminal, which is one of Velocity's core value propositions.

Working Directory Detection

Terminals detect CWD through:

1. OSC 7 escape sequence (shell reports CWD)
2. /proc filesystem queries (Linux)
3. libproc queries (macOS)

VS Code's implementation is the gold standard: clicking links resolves relative to the CWD of the command that produced them.

Clipboard Integration (OSC 52)

Allows terminal applications to read/write the system clipboard. Critical for:

- Copy from tmux/Vim to system clipboard
- Remote clipboard sync over SSH

Support varies: Kitty (write by default, read opt-in), iTerm2 (write only, must enable), Alacritty (supported), GNOME Terminal (no support).

2.5 Modern and Power Features

AI Integration

Warp leads this category. Four interaction modes:

1. Traditional command line
2. AI completions (type-ahead suggestions)
3. Interactive chat (ask questions, get commands)
4. Autonomous/agent mode (AI executes multi-step tasks)

The broader trend: AI CLI tools (Claude Code, Gemini CLI) operate inside any terminal. The terminal itself may not need to embed AI - instead, it should be an excellent host for AI CLI tools.

Recommendation for Velocity: Do not embed AI in the terminal. Instead, ensure our terminal is an excellent host for AI tools: support modern escape sequences, have fast rendering for AI agent output, support semantic zones for AI-generated content.

Command Palette

Warp and Ghostty both offer command palettes. This aligns perfectly with Velocity's Cmd+Shift+P design.

tmux Integration

iTerm2's control mode (-CC) is the gold standard:

- tmux panes render as native tabs/splits
- Native scrollbars replace tmux status line
- Mouse and keyboard shortcuts work naturally
- Can reconnect with standard tmux from another client

This is a massive productivity win and very few terminals support it. Supporting tmux control mode would be a significant competitive advantage.

Triggers and Automation

iTerm2's triggers: regex-matched rules that fire when output matches patterns. Actions include:

- Highlight text
- Show notification
- Run command
- Set terminal title
- Send text to terminal
- Capture to toolbelt

This feature is unique to iTerm2 and frequently cited as a reason developers stay.

PART 3: WHAT MAKES A TERMINAL "FEEL FAST"

3.1 Input Latency Architecture

The critical pipeline from keypress to rendered character:

1. Hardware input event (keyboard -> USB -> OS)
2. OS event delivery to application
3. Application processing (escape sequence generation)
4. PTY write (send to shell)
5. Shell processing (echo, if applicable)
6. PTY read (receive from shell)
7. Escape sequence parsing
8. Grid/buffer update
9. Rendering decision (should we repaint now?)
10. GPU/CPU rendering
11. Display compositor
12. Monitor refresh

Steps 7-10 are where terminal emulators differentiate. Key optimizations:

- Threaded I/O: Kitty separates child process I/O from rendering (steps 6-8 in one thread, 9-10 in another)
- Non-blocking PTY reads: Use kqueue/epoll for I/O notification, never block the render thread
- SIMD parsing: Kitty uses vector CPU instructions for escape sequence parsing
- VRAM glyph cache: Pre-render glyphs to GPU texture, reuse for subsequent frames
- Batched rendering: Coalesce multiple updates into a single frame

3.2 Frame Pacing and Adaptive Rendering

The fundamental tradeoff: latency vs. throughput vs. flicker

- Smaller `repaint_delay` = lower latency, but more CPU usage and potential flicker
- Larger `repaint_delay` = better throughput (more data per frame), less flicker, but higher latency

Kitty's approach:

- `repaint_delay`: configurable (default balances latency and efficiency)
- `input_delay`: batches rapid input to reduce render calls
- `sync_to_monitor`: optionally align renders with monitor vsync

Best practice for embedded terminal:

- Under normal typing: minimize latency (render ASAP after each keystroke)
- Under high output flood (e.g., `cat large_file`): switch to throughput mode (render only visible portion of each frame,

skip intermediate frames)

- Use synchronized rendering protocol: applications can signal "start batch" / "end batch" to prevent partial frame rendering

3.3 Scrollback Search Performance

Large scrollback buffers (100K+ lines) create search challenges:

- Linear scan is too slow for regex search
- Ring buffers cap memory but lose history
- Persistent scrollback (written to disk) enables unlimited history but requires indexed search

Approach for Velocity: Use a ring buffer in memory with configurable size. For search, maintain a lightweight index. For persistent scrollback, write to disk with SQLite FTS5 (consistent with our search index architecture).

3.4 Perceived Smoothness

Users judge "fast" by:

1. Input responsiveness: Does typing feel instant? (< 8ms target)
2. Scroll smoothness: Is scrolling 60fps with no jank? (frame-time variance < 2ms)
3. Output flood handling: Does the terminal stay responsive during heavy output?
4. Startup time: Is it ready immediately? (< 1 second cold)
5. Search speed: Do results appear instantly? (< 100ms for first result)

PART 4: EMERGING TRENDS (2024-2026)

4.1 AI Integration Is the Dominant Trend

- Warp evolved from terminal to "Agentic Development Environment"
- Wave Terminal integrates AI chat with file management
- AI CLI tools (Claude Code, Gemini CLI, GitHub Copilot CLI) work inside any terminal
- The winner may not be "AI inside the terminal" but "terminal that's best for AI tools"

4.2 GPU Rendering Is Now Table Stakes

- Every new terminal uses GPU rendering
- CPU-rendered terminals (iTerm2, Terminal.app) are seen as legacy
- Metal on macOS is superior to OpenGL for native feel

4.3 Kitty Graphics Protocol Is Becoming the Standard

- Supported by: Kitty, Ghostty, WezTerm (and growing)
- CLI tools (tmux, chafa, yazi) increasingly default to Kitty protocol
- Sixel is declining as the "modern" choice despite broader legacy support

4.4 Shell Integration Is Increasingly Expected

- OSC 133 semantic prompt marks are widely adopted
- Working directory tracking enables rich IDE-like features
- Prompt navigation transforms scrollback usability

4.5 Native Platform Integration Matters More Than Cross-Platform

- Ghostty's success proves developers prefer "feels like a Mac app" over "works on every OS"
- Platform-specific features (Metal, Core Text, macOS Secure Input, Quick Look) are valued
- This strongly supports Velocity's AppKit-first approach

4.6 Block-Based Output Is Being Explored

- Warp pioneered treating each command as a discrete block
- Makes it easy to: copy specific command output, share results, navigate history

- However, departs from traditional terminal UX, which some developers resist

4.7 Configuration-as-Code Is Preferred

- WezTerm's Lua scripting enables dynamic, programmatic configuration
- TOML/YAML files are preferred over GUI preferences
- Hot-reload of configuration is expected (change config, see results immediately)

4.8 Privacy and Open Source Are Increasingly Valued

- Warp's telemetry and account requirement are frequent complaints
- Open source terminals (Ghostty, Kitty, Alacritty, WezTerm) dominate developer preference
- Developers resist terminals that phone home

4.9 Terminal Multiplexer Integration

- tmux remains dominant for session management
- iTerm2's control mode is the only deep integration
- Zellij is gaining as a modern tmux alternative
- Some terminals build multiplexing in (WezTerm, Kitty) rather than integrating tmux

PART 5: EMBEDDED TERMINALS - SPECIFIC INSIGHTS

5.1 What Makes Embedded Terminals Succeed or Fail

Success Factors

1. Low latency that matches standalone terminals -- If the embedded terminal feels slower than iTerm2, developers will Alt-Tab to iTerm2 and ignore the embedded one.
2. Working directory synchronization -- The killer feature: navigate to a folder in the file manager, terminal CWD follows automatically. And vice versa.
3. Path insertion from file pane -- Drag a file from the file browser to the terminal, get a properly escaped path inserted at cursor position.
4. Doesn't steal keyboard focus -- Keyboard shortcuts must be cleanly partitioned between file manager and terminal. No conflicts.
5. Resizable and hideable -- Users must be able to resize the terminal panel and dismiss it when not needed. Toggle with a keyboard shortcut.
6. Multiple sessions -- Tabs or splits within the embedded terminal.
7. Shared clipboard -- Copy from file pane, paste in terminal, and vice versa.

Failure Factors

1. Sluggish performance -- If embedded terminal is noticeably slower than standalone, it's dead on arrival.
2. Limited terminal emulation -- Must handle vim, htop, tmux, ssh sessions without glitches.
3. Broken escape sequences -- ncurses applications must render correctly.
4. No search -- Users expect to search terminal scrollback.
5. No customization -- Users expect control over font, color scheme, keybindings.
6. Interfering with focus -- If file manager shortcuts fire while typing in terminal, UX is broken.

5.2 Path Finder's Terminal Integration - User Feedback

Based on available reviews:

- The terminal panel at the bottom of Path Finder is appreciated as a concept
- However, Path Finder's overall stability issues (crashes, lost configurations, UI glitches) have undermined trust in all its features, including the terminal
- Users report the terminal module as "useful when it works" but unreliable
- The key lesson: an embedded terminal's reputation depends on the host application's stability

Key takeaway: Our terminal module must be rock-solid. A crash in the terminal panel reflects on all of Velocity.

5.3 VS Code's Integrated Terminal - Lessons Learned

VS Code's terminal is the most successful embedded terminal in any application. Key lessons:

1. Shell integration is the foundation: VS Code injects invisible shell integration scripts that enable:
 - Working directory detection (links resolve relative to the CWD of the command that produced them)
 - Command detection (decorations appear next to commands, navigation between prompts)
 - Clickable file paths that open in the editor
 - Terminal suggestions (IntelliSense-style completions for shell commands)
2. Multi-terminal support: Multiple terminal instances with easy switching. Split terminals. Terminal groups.
3. CWD inheritance: New terminals open in the workspace folder. New splits inherit the CWD of the source terminal.
4. Editor integration: Error detection in terminal output with one-click navigation to the file/line. This is extremely valuable.
5. Profile system: Users configure different shell profiles (bash, zsh, PowerShell) with different settings.
6. Keyboard shortcut partitioning: `Ctrl+`` toggles the terminal panel. When the terminal has focus, most VS Code shortcuts are passed through to the terminal. This is carefully designed to avoid conflicts.
7. Terminal rendering: VS Code uses `xterm.js` (JavaScript-based terminal renderer in a web view). Performance is acceptable but not competitive with native terminals. This is the weak point of VS Code's terminal.

Key takeaway for Velocity: Adopt VS Code's shell integration model (OSC 133 support, CWD tracking, clickable paths that open in the file viewer). But use native rendering (SwiftTerm + Metal/AppKit) instead of web-based rendering.

5.4 File-to-Terminal Workflow Productivity

Directory Synchronization (CD Sync)

The most requested feature for file manager + terminal integration:

File Pane -> Terminal: When user navigates to a directory in the file browser, automatically ``cd`` the terminal to that directory.

Terminal -> File Pane: When user ``cd``s in the terminal, update the file browser to show that directory.

Implementation considerations:

- Use OSC 7 (working directory reporting) or `/proc/libproc` queries for Terminal -> File Pane direction
- For File Pane -> Terminal direction, send ``cd "/path/to/dir"\n`` to the PTY -- but only if the terminal is at a shell prompt (not running a program). Use shell integration marks to detect this.
- Make this behavior toggleable (some users find auto-sync disruptive)
- Visual indicator when sync is active/inactive

Drag-and-Drop Path Insertion

Critical workflow: drag file(s) from file browser into terminal to insert their paths.

Requirements (from our quality gates):

- Shell-safe escaping: spaces, quotes, unicode, shell metacharacters
- Multi-file drag inserts space-delimited escaped paths
- Deterministic ordering
- Modifier key for relative path insertion (relative to terminal CWD)
- Works from both internal file panes and external Finder drags
- Insertion at cursor position (not appended to end)
- Preview of insertion before drop commit
- Latency: < 50ms from drop to inserted text

Implementation: This requires PTY-level cursor position awareness and the ability to inject text at the cursor. Use bracketed paste mode (OSC 200/201) for safe injection that doesn't trigger shell completion or execution.

Related Workflows

- Open terminal at folder: Right-click a folder in file browser, "Open Terminal Here"
- Copy path: Copy file/folder path to clipboard from file browser, paste in terminal
- Run command on selection: Select files in browser, run a command template on them
- Reveal in file browser: From terminal output, click a file path to reveal it in the file pane

5.5 Recommendations for Velocity's Embedded Terminal

Based on all research, here are the prioritized recommendations:

Must Implement (Phase 1 - MVP)

1. SwiftTerm with PTY for terminal emulation
2. Metal-accelerated rendering (or use AppKit's NSTextView with hardware layers)
3. Keystroke-to-echo latency target: < 8ms (p95)
4. Basic tabs (horizontal)
5. Shell integration: OSC 7 (CWD), OSC 133 (semantic prompts)
6. Bidirectional directory sync between file pane and terminal
7. Drag-and-drop file path insertion with shell-safe escaping
8. Search in scrollback (regex support)
9. True color (24-bit) and 256-color support
10. Unicode support (emoji, combining characters, CJK)
11. Clickable URLs (OSC 8 hyperlinks)
12. Configurable font with Nerd Font fallback
13. Resizable terminal panel with keyboard toggle
14. Focus management: clean keyboard shortcut partitioning between file manager and terminal
15. Session state persistence across app restarts

Should Implement (Phase 2-3)

16. Vertical splits within terminal panel
17. Kitty Graphics Protocol (enables image previews in terminal)
18. Font ligature support
19. Shell integration: prompt-to-prompt navigation
20. Clickable file paths that reveal in file browser or open in viewer
21. Profile management (per-project terminal settings)
22. OSC 52 clipboard integration
23. Scrollback persistence to disk
24. Terminal notifications (bell, process completion)
25. Theme system with dark/light mode auto-switching
26. Broadcast input to multiple terminal panes

Nice to Have (Phase 4-5)

27. tmux control mode integration
28. Triggers/automation (regex-matched actions)
29. Block-based output mode (optional, Warp-inspired)
30. iTerm2 image protocol support (for broader compatibility)
31. Sixel support (for legacy tool compatibility)
32. Terminal recording/playback (asciinema integration)
33. SSH integration with connection manager
34. Serial port support (for hardware/embedded development)
35. Python/Lua scripting API for terminal automation

PART 6: TERMINAL COMPATIBILITY MATRIX

Feature Support Across Major Terminals

Feature	Alacritty	iTerm2	Kitty	WezTerm	Ghostty	Hyper	Termina..	Warp
GPU Rendering	Yes (OGL)	No	Yes (OGL)	Yes (OGL)	Yes (Me..	Partial	No	Yes
True Color	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Font Ligatures	No	Partial	Yes	Yes	Yes	Partial	No	Yes
Tabs	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Split Panes	No	Yes	Yes	Yes	Yes	Yes	No	Yes
Sixel Graphics	No	Yes	No	Yes	No	No	No	No
Kitty Graphics	No	No	Yes	Yes	Yes	No	No	No
iTerm2 Images	No	Yes	No	Yes	No	No	No	No
Shell Integration	No	Yes	Yes	Yes	Yes	No	No	Yes
OSC 52 Clipboard	Yes	Yes	Yes	Yes	Partial	No	No	Yes
Clickable URLs	Yes	Yes	Yes	Yes	Yes	Partial	No	Yes
tmux Control Mode	No	Yes	No	No	No	No	No	No
Triggers	No	Yes	No	No	No	No	No	No
Command Palette	No	No	No	Yes	Yes	No	No	Yes
AI Integration	No	Partial	No	No	No	No	No	Yes
Broadcast Input	No	Yes	Yes	No	No	No	No	No
Password Manager	No	Yes	No	No	No	No	No	No
Desktop Notif.	No	Yes	Yes	Yes	Yes	No	No	Yes
Memory (idle)	~30MB	~180MB	~50MB	~60MB	~35MB	~200MB	~40MB	~100MB
Cross-Platform	Yes	No	Yes	Yes	No*	Yes	No	Yes

*Ghostty: macOS + Linux only (no Windows)

PART 7: STRATEGIC RECOMMENDATIONS FOR VELOCITY

7.1 Competitive Positioning

Velocity's embedded terminal should aim to be:

- As fast as Alacritty (input latency)
- As feature-rich as iTerm2's shell integration (triggers, profiles, tmux mode)
- As native-feeling as Ghostty (Metal rendering, macOS conventions)
- With the file integration of Wave Terminal (but much faster)
- With the directory sync of VS Code's terminal (CWD tracking, path clicking)

This is achievable because we have a narrower scope (embedded terminal, not standalone) and we're building on SwiftTerm (proven library used in commercial SSH clients).

7.2 Architecture Recommendations

1. Rendering: Use Metal via SwiftTerm's AppKit view, with VRAM glyph caching inspired by Kitty
2. I/O Threading: Separate PTY I/O thread from render thread (Kitty's architecture)
3. Frame Coalescing: Under high output, coalesce frames to maintain responsiveness (render only visible portion of last frame worth of data)
4. Scrollback: Ring buffer in memory (configurable size) + optional disk persistence via SQLite
5. Escape Parsing: Optimize hot path with SIMD where applicable (Kitty's approach)
6. Shell Integration: Auto-inject integration scripts for Bash/Zsh/Fish on terminal launch (VS Code's approach)

7.3 Key Differentiators to Pursue

1. Bidirectional directory sync -- Navigate files, terminal follows. `cd` in terminal, files follow.
2. Drag-to-terminal path insertion -- The smoothest file-to-command workflow on macOS
3. Click-to-reveal -- Click any file path in terminal output to reveal in file browser or open in built-in viewer
4. File operation integration -- Terminal commands that produce file changes are reflected in real-time in the file browser (via FSEvents)
5. Run on selection -- Select files in browser, type a command template that executes on all selected files
6. Terminal-aware file operations -- F5 copy, F6 move, F7 mkdir work in both file panes AND terminal context

7.4 What NOT to Do

1. Do not embed AI directly -- Let AI CLI tools (Claude Code, etc.) run inside our terminal. Keep the terminal a clean rendering surface.
2. Do not require accounts or telemetry -- This is a primary complaint about Warp. Respect privacy.
3. Do not use Electron or web views for terminal rendering -- Hyper and Tabby prove this is a dead end.
4. Do not sacrifice performance for features -- Every feature must meet the 8ms latency bar.
5. Do not break standard terminal emulation -- vim, htop, tmux, ssh must work perfectly.
6. Do not ignore tmux users -- Many developers live in tmux. Our terminal must work excellently as a tmux host, and ideally support control mode.

APPENDIX A: SWIFTTERM CAPABILITIES ASSESSMENT

SwiftTerm (github.com/migueldeicaza/SwiftTerm) is our chosen terminal emulation library. Assessment:

Strengths:

- VT100/Xterm emulation quality: "on par or better than XtermSharp and xterm.js"
- Proper Core Text rendering for Unicode test suites
- Reusable engine architecture (UI-agnostic core)
- Selection engine with macOS support
- Emoji and combining character support
- Used in commercial products (Secure Shellfish, La Terminal)
- Swift Package Manager integration
- asciinema recording/playback support

Gaps to Evaluate:

- GPU rendering support (may need custom Metal layer on top)
- Performance under high throughput vs Kitty/Alacritty
- Kitty Graphics Protocol support (likely needs implementation)
- Shell integration (OSC 133) support level
- Font ligature handling
- Sixel/image protocol support

Recommendation: SwiftTerm provides a solid emulation foundation. We likely need to build a custom Metal rendering layer on top of it (rather than using its default AppKit/SwiftUI views) to achieve our latency targets. The Kitty Graphics Protocol and advanced shell integration may need to be implemented as extensions.

APPENDIX B: SOURCES

Terminal Emulator Official Sites

- Ghostty: <https://ghostty.org/docs/features>
- Kitty: <https://sw.kovidgoyal.net/kitty/>
- Alacritty: <https://alacritty.org/>
- Warp: <https://www.warp.dev/all-features>
- WezTerm: <https://wezterm.com/>
- iTerm2: <https://iterm2.com/features.html>
- Wave Terminal: <https://www.waveterm.dev/>
- Rio: <https://rioterm.com/>
- Tabby: <https://tabby.sh/>
- SwiftTerm: <https://github.com/migueldeicaza/SwiftTerm>

Research and Comparison Articles

- State of Terminal Emulators in 2025: <https://www.jeffquast.com/post/state-of-terminal-emulation-2025/>
- Terminal Compatibility Matrix: <https://tmuxai.dev/terminal-compatibility/>
- Best Terminal Emulators 2026 (Scopir): <https://scopir.com/posts/best-terminal-emulators-developers-2026/>
- Choosing a Terminal on macOS 2025: <https://medium.com/@dynamicy/choosing-a-terminal-on-macos-2025-iterm2-vs-ghostty-vs-wezterm-vs-kitty-vs-alacritty-d6a5e42fd8b3>
- Linux Terminal Emulator Statistics 2026: <https://commandlinux.com/statistics/linux-terminal-emulator-popularity-statistics/>
- Modern Terminals Showdown: <https://blog.codeminer42.com/modern-terminals-alacritty-kitty-and-ghostty/>

Performance and Technical References

- Kitty Performance: <https://sw.kovidgoyal.net/kitty/performance/>
- Warp Performance Benchmarks: <https://docs.warp.dev/terminal/comparisons/performance>
- Terminal Latency (Dan Luu): <https://danluu.com/term-latency/>
- Terminal Latency (beuke.org): <https://beuke.org/terminal-latency/>
- Hyperlinks in Terminal Emulators (OSC 8): <https://gist.github.com/egmontkob/eb114294efbcd5adb1944c9f3cb5feda>
- Are We Sixel Yet: <https://www.aresixelyet.com/>
- Kitty Graphics Protocol: <https://sw.kovidgoyal.net/kitty/graphics-protocol/>
- OSC 133 Shell Integration: <https://github.com/tmux/tmux/issues/3064>

VS Code Terminal

- Shell Integration: <https://code.visualstudio.com/docs/terminal/shell-integration>
- Terminal Basics: <https://code.visualstudio.com/docs/terminal/basics>
- Terminal Advanced: <https://code.visualstudio.com/docs/terminal/advanced>

Developer Discussions

- HackerNews: State of Terminal Emulators: <https://news.ycombinator.com/item?id=45799478>
- Ghostty switching discussions: <https://github.com/ghostty-org/ghostty/discussions/4837>
- Warp accessibility: <https://github.com/warpcatdev/Warp/discussions/1704>
- tmux control mode: <https://iterm2.com/documentation-tmux-integration.html>

This report was compiled on March 7, 2026 for the Velocity project. Information reflects the state of terminal emulators as of early 2026.